

Release Engine - README

Release Engine Workload Patterns (Template Repository)

- Repository Overview
- Architecture Context
 - This Repository's Role
 - Current Patterns
 - Available Deployment Patterns
- Pattern Structure
 - Pattern Documentation
- Quick Start (Using as Template)
 - Step 1: Clone This Template
 - Step 2: Set Up Upstream Tracking
 - Step 3: Customize for Your Organization
- Creating New Patterns
- Pattern Development Process
- Infrastructure as Code Guidelines
- Pipeline Configuration (`workload.yml`)
- Pattern Categories
 - Platform Patterns
 - Application Patterns
 - Data Patterns
- Template Maintenance
 - Keeping Up-to-Date with Upstream
 - Contributing Back to Template
- Testing and Validation
 - Pattern Testing Strategy
- Quality Gates
- Documentation Standards
 - Pattern Documentation Template
- Integration with Release Engine
- Repository Dependencies
- Usage in Configuration Repositories
- Best Practices
 - Development Guidelines
 - Security Considerations
 - Performance Optimization
- Support and Resources
 - Getting Help
 - Related Resources

Release Engine Workload Patterns (Template Repository)

Download this documentation: [PDF](#) | [DOCX](#) | [All Downloads](#)

Repository Overview

This repository serves as the **Abstraction Layer Template** in the Release Engine three-tier solution. It contains reusable workload patterns that define "how" infrastructure and applications are deployed through Infrastructure as Code (Bicep) templates and pipeline configurations.

Important: This is a **template repository**. Organizations should clone this repository to create their own workload pattern collections.

Architecture Context

Configuration Layer (Simple) → Abstraction Layer (THIS REPO) → Core Layer (Pipelines)

This Repository's Role

- **Pattern Library:** Contains proven deployment patterns for common workload types
- **Infrastructure as Code:** Azure Bicep templates with Azure Verified Modules integration
- **Pipeline Orchestration:** Workload-specific pipeline configurations and dependency management
- **Template Foundation:** Starting point for organizations to build their pattern collections

Current Patterns

Available Deployment Patterns

Pattern	Description	Complexity	Deployer Scope
resource_group_scope_pattern	Simple single-resource deployments within a resource group	Basic	Resource Group
subscription_scope_pattern	Subscription-level resource deployments	Intermediate	Subscription
multi_stage_pattern	Complex multi-stage deployments with dependencies	Advanced	Subscription

Pattern Structure

Each pattern directory contains:

```
patterns/{pattern_name}/
├── workload.yml           # Pipeline configuration and orchestration
├── {pattern_name}.bicep   # Main infrastructure template
├── {pattern_name}.prerequisite.bicep # Prerequisites (if multi-stage)
├── {pattern_name}.dependent.bicep   # Dependent resources (if multi-stage)
└── README.md             # Pattern documentation and usage guide
```

Pattern Documentation

- **Comprehensive Patterns Overview** - Detailed comparison, selection guide, and technical architecture
- **Individual Pattern READMEs** - Complete usage instructions, parameters, and examples for each pattern
- **AI Assistant Instructions** - Detailed guidance for AI agents working with patterns
- **Documentation Downloads** - Download all documentation in PDF and DOCX formats

Quick Start (Using as Template)

Step 1: Clone This Template

```
# Clone this template repository
git clone https://github.com/thecloudexplorers/release-engine-
example-workload-pattern.git

# Rename for your organization
mv release-engine-example-workload-pattern release-engine-myorg-
workload-patterns
cd release-engine-myorg-workload-patterns
```

Step 2: Set Up Upstream Tracking

```
# Add upstream remote for future updates
git remote add upstream
https://github.com/thecloudexplorers/release-engine-example-
workload-pattern.git

# Set new origin (your repository)
git remote set-url origin https://github.com/myorg/release-engine-
myorg-workload-patterns.git

# Verify configuration
git remote -v
```

Step 3: Customize for Your Organization

1. **Update Service Connections:** Modify service connection references in workload.yml files
2. **Add Organization Patterns:** Create patterns specific to your organization's needs
3. **Update Documentation:** Customize READMEs and pattern documentation
4. **Test Patterns:** Validate patterns work with your Azure environment

Creating New Patterns

Pattern Development Process

1. Create Pattern Directory

```
mkdir patterns/my_new_pattern
cd patterns/my_new_pattern
```

2. Required Files

- workload.yml - Pipeline configuration and deployment

- orchestration
- {pattern_name}.bicep - Infrastructure as Code definitions
- README.md - Pattern documentation and usage instructions

3. Follow Naming Conventions

- Use descriptive names: webapp_with_database, function_app_premium
- Use underscores for separation
- Keep names concise but clear

Infrastructure as Code Guidelines

Azure Bicep Best Practices

```

metadata resources = {
  version: '0.1.0'
  author: '<Your Name>'
  company: '<Your Organization>'
  description: '<Pattern Description>'
}

targetScope = 'subscription' // or 'resourceGroup'

@allowed([
  'westeurope'
  'uksouth'
  'eastus'
])
@description('Region in which resources should be deployed')
param resourceLocation string

param tags object = {}

```

Azure Verified Modules Priority

- **Always check AVM first:** Use br/public:avm/res/ registry
- **Pin versions:** Always specify exact AVM versions
- **⚠ Document exceptions:** If AVM unavailable, document why using direct resources

Example AVM Usage:

```

module storageAccount 'br/public:avm/res/storage/storage-account:0.27.1' = {
  name: 'storageAccountDeployment'
  params: {
    name: storageAccountName
    location: resourceLocation
    skuName: storageAccountSku
    tags: tags
  }
}

```

Pipeline Configuration (workload.yml)

Basic Pattern Template

```

parameters:
  - name: deploymentSettings
    type: object

variables:
  - name: serviceConnection

```

```

        value: <your-org-service-connection>

    stages:
      - template: /pipelines/01-
        orchestrators/pattern.orchestrator.yml@release-engine-core
        parameters:
          patternSettings:
            name: <pattern_name>
            configurationFilePath: ${
parameters.deploymentSettings.configurationFilePath }}
            environments: ${ parameters.deploymentSettings.environments
}}

          patternArtifactsPath: /patterns/<pattern_name>
          stages:
            - infrastructure:
                iac:
                  name: <deployment_name>
                  displayName: <human_readable_name>
                  deploymentScope: <Subscription|ResourceGroup>
                  serviceConnection: $(serviceConnection)
                  iacMainFileName: <bicep_file>.bicep
                  iacParameterFileName: ${
parameters.deploymentSettings.iacParameterFileName }}

```

Multi-Stage Pattern Example

For complex deployments with dependencies:

```

stages:
  # Prerequisites (no dependencies)
  - infrastructure:
      iac:
        name: prerequisite_stage
        displayName: Prerequisites
        deploymentScope: Subscription
        iacMainFileName: prerequisite.bicep

  # Dependent deployment (depends on prerequisites)
  - infrastructure:
      iac:
        name: main_deployment
        displayName: Main Infrastructure
        deploymentScope: ResourceGroup
        iacMainFileName: main.bicep
        dependsOn: prerequisite_stage
        lastInStage: true

```

Pattern Categories

Platform Patterns

Infrastructure foundations and shared services:

- **Logging Infrastructure:** Centralized logging with Log Analytics
- **Monitoring Infrastructure:** Azure Monitor and Application Insights
- **Networking Hub:** Hub network with firewall and gateway
- **Security Baseline:** Key Vault and security configurations

Application Patterns

Application-specific deployment patterns:

- **Web App Basic:** Simple web app with app service plan
- **Web App with Database:** Web app with SQL database integration
- **Function App Premium:** Premium function app with dependencies
- **Container App:** Containerized application deployment

Data Patterns

Data platform and analytics patterns:

- **SQL Database:** SQL Database with backup and security
- **Cosmos Database:** Cosmos DB with consistency configurations
- **Synapse Workspace:** Analytics workspace and pools
- **Data Factory:** ETL pipelines and data integration

Template Maintenance

Keeping Up-to-Date with Upstream

Regular Synchronization (Recommended: Monthly)

```
# Fetch upstream changes
git fetch upstream

# Create integration branch
git checkout -b upstream-sync-$(date +%Y%m%d)

# Merge upstream improvements
git merge upstream/main

# Resolve conflicts (preserve your customizations)
# Test changes thoroughly
# Push and create PR to main
```

What to Sync

- **New Patterns:** Additional patterns that benefit your organization
- **Bug Fixes:** Fixes to existing patterns and pipeline configurations
- **Security Updates:** Security improvements and best practices
- **Documentation:** Improved examples and documentation

What to Preserve

- **Service Connections:** Your organization-specific service connections
- **Custom Patterns:** Your organization-specific deployment patterns
- **Naming Conventions:** Your organization's naming standards
- **Customizations:** Any modifications for your environment

Contributing Back to Template

When to Contribute

- **Universal Patterns:** Patterns that would benefit multiple organizations
- **Bug Fixes:** Fixes to existing patterns or configurations
- **Documentation:** Improvements to pattern documentation
- **Enhancements:** Performance or security improvements

Contribution Process

1. **Fork** the upstream repository
2. **Create** feature branch: feature/new-pattern-name
3. **Develop** and test your improvements
4. **Document** changes and provide examples
5. **Submit** pull request with clear description

Testing and Validation

Pattern Testing Strategy

Local Validation

```
# Validate Bicep syntax
az bicep build --file patterns/pattern_name/pattern.bicep

# Validate pipeline YAML
az pipelines validate --yaml-path patterns/pattern_name/workload.yml
```

Integration Testing

1. **Deploy in Isolation:** Test patterns in isolated test environments
2. **Validate Outputs:** Ensure all expected outputs are generated
3. **Test Dependencies:** Verify multi-stage dependencies work correctly
4. **Check Configuration:** Validate parameter files work with patterns

Quality Gates

- **Bicep Validation:** All templates must compile without errors
- **Parameter Validation:** All required parameters must be documented
- **Output Documentation:** All outputs must be documented
- **Pattern Documentation:** Each pattern must have comprehensive README

Documentation Standards

Pattern Documentation Template

Each pattern should include a comprehensive README:

```
# [Pattern Name] Pattern

## Overview
Brief description of what this pattern deploys.

## Architecture
Description or diagram of deployed resources.

## Prerequisites
- Required permissions
- Dependencies
- Service principal requirements

## Parameters
| Parameter | Type | Description | Default | Required |
|-----|-----|-----|-----|-----|
```

```
| param1 | string | Description | value | Yes |
```

Outputs

List of outputs and their purposes.

Usage Example

Reference to configuration setup.

Dependencies

Any pattern dependencies.

Integration with Release Engine

Repository Dependencies

This repository integrates with:

- **Release Engine Core:** Provides pipeline orchestration framework
- **Configuration Repositories:** Uses patterns defined here for actual deployments

Usage in Configuration Repositories

Configuration repositories reference patterns from this repository:

```
# In configuration repository azure-pipelines.yml
resources:
  repositories:
    - repository: workload
      type: github
      name: myorg/release-engine-myorg-workload-patterns
      endpoint: myorg-github
      ref: refs/heads/main

extends:
  template: /patterns/webapp_basic/workload.yml@workload
```

Best Practices

Development Guidelines

- **Start Simple:** Begin with basic patterns, add complexity gradually
- **Test Thoroughly:** Validate patterns in non-production environments first
- **Document Everything:** Comprehensive documentation enables self-service adoption
- **Follow Conventions:** Consistent naming and structure across patterns

Security Considerations

- **Least Privilege:** Use minimal required permissions for deployments
- **Managed Identities:** Prefer system-assigned managed identities
- **Secure Defaults:** Configure resources with security-first defaults
- **Compliance:** Ensure patterns meet organizational compliance requirements

Performance Optimization

- **Parallel Deployment:** Structure patterns to enable parallel resource creation
- **Resource Dependencies:** Minimize unnecessary dependencies between resources
- **Deployment Speed:** Optimize for fast deployment times
- **Error Recovery:** Design patterns for easy rollback and recovery

Support and Resources

Getting Help

- **Documentation:** Comprehensive guides in [AGENTS.md](#)
- **Community:** GitHub Discussions for questions and ideas
- **Issues:** Report bugs via GitHub Issues
- **Contributions:** Follow contribution guidelines for improvements

Related Resources

- **Release Engine Core:** Core pipeline framework
- **Configuration Template:** Configuration repository template
- **Azure Bicep Documentation**
- **Azure Verified Modules**

This template repository enables organizations to create sophisticated workload deployment patterns while maintaining consistency and leveraging community improvements. Clone, customize, and extend to meet your organization's specific needs.